ROBENDEVS FOUNDATION PROGRAMME · DAYS 1–2

https://robendevs.com ⏎

What happens when you press Enter?

Internet, networking, HTTP/HTTPS and browser internals — the basics

Software Engineering Internship · Trainer: Mehedi Hasan Shipon · November 2026

TODAY'S MAP

Six stops on the journey of one request

01

Networking basics

IP, ports, layers, TCP vs UDP

02

DNS

Turning a name into an IP address

03

HTTP

Requests, responses, status codes, versions

04

HTTPS & TLS

Encryption, certificates, the handshake

05

Browser internals

Processes, DOM, CSSOM, rendering

06

Caching & security

Cache headers, cookies, CORS, XSS, CSRF

About 90 minutes · questions any time · no coding needed — just curiosity

THE BIG PICTURE

The request lifecycle in seven steps

1
Type URL
Browser parses it: scheme, host, path

2
DNS
Name → IP address

3
TCP
Open a reliable connection

4
TLS
Agree on secret keys

5
Request
GET / with headers

6
Response
Server sends HTML, CSS, JS

7
Render
Browser paints pixels

Every step takes time, and every step can fail. Debugging the web = knowing **which step** broke.

01 · NETWORKING BASICS

IP addresses and ports: building and flat number

IP address

Finds the **machine**. Like a building's address.

```
IPv4 203.0.113.10
IPv6 2001:db8::10
```

Port

Finds the **program** on that machine. Like a flat number.

```
80 HTTP · 443 HTTPS
22 SSH · 5432 Postgres
```

Private vs public

Your laptop has a private IP. The router shares one public IP for the whole office using **NAT**.

```
192.168.0.105 → router → public IP
```

A connection is identified by: source IP + source port + destination IP + destination port + protocol.

The TCP/IP model: each layer has one job

Layer	Its job	Examples	Post-office analogy
Application	What the message says	HTTP, DNS, SMTP	The letter itself
Transport	Deliver to the right app, reliably or fast	TCP, UDP, ports	Registered post vs flyer
Internet	Route packets across networks	IP, routers	Address and sorting centre
Link	Move bits over one physical link	Ethernet, Wi-Fi	The delivery van

The **OSI model** splits this into 7 layers (it adds Session and Presentation, and splits Link into Data Link and Physical). Interviews say “L4” and “L7” — that’s OSI numbering: **L4 = Transport**, **L7 = Application**.

01 · NETWORKING BASICS

TCP vs UDP: reliable or fast

TCP

- Connection first (handshake)
- Ordered, nothing lost — resends missing packets
- Slower to start

Used by: HTTP/1.1, HTTP/2, SSH, databases

UDP

- No connection, just send
- No order or delivery guarantee
- Very low overhead

Used by: DNS, video calls, games, HTTP/3 (QUIC)

3-way handshake

`Client → SYN`

`Server → SYN-ACK`

`Client → ACK`

One full round trip before any data. Dhaka ↔ a US server is a long trip, so this cost adds up.

DNS: the internet’s phone book

1. **Browser cache** — visited recently?
2. **OS cache** and the hosts file
3. **Recursive resolver** — ISP, 1.1.1.1 or 8.8.8.8
4. **Root server** — “ask the .com servers”
5. **TLD server (.com)** — “ask robendevs.com’s name servers”
6. **Authoritative server** — gives the real answer

THE ANSWER

```
robendevs.com
A 203.0.113.10
TTL 300
```

The resolver caches it for 300 seconds. The next lookup skips steps 4–6 and takes about 1 ms.

Record types you will meet at work

Type	Points to	Example
A	An IPv4 address	robendevs.com → 203.0.113.10
AAAA	An IPv6 address	robendevs.com → 2001:db8::10
CNAME	Another name (alias)	www.robendevs.com → robendevs.com
MX	The mail server	Where @robendevs.com email goes
TXT	Free text	SPF for email, domain ownership checks
NS	Who is authoritative	The domain's name servers

Production story: we move to a new server, but users still hit the old IP for hours. Why? The old record had a long TTL. **Lower the TTL a day before a migration.**

An HTTP message is just structured text

REQUEST

```
GET /about HTTP/1.1
Host: robendevs.com
User-Agent: Chrome/…
Accept: text/html
Cookie: session=abc123
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: text/html
Cache-Control: max-age=600
Set-Cookie: theme=dark

<!doctype html>…
```

Start line — method + path, or status code

Headers — metadata as key: value

Body — the content, after a blank line

Stateless — cookies give it memory

Methods say what to do; status codes say how it went

Methods

Method	Meaning
GET	Read data (safe, no changes)
POST	Create something new
PUT	Replace a whole resource
PATCH	Update part of a resource
DELETE	Remove a resource

Status codes

Class	Meaning · common codes
2xx	Success · 200, 201, 204
3xx	Go elsewhere · 301, 302, 304
4xx	Client mistake · 400, 401, 403, 404, 429
5xx	Server problem · 500, 502, 503, 504

401 vs 403: 401 = “who are you?” (not logged in). 403 = “I know you, but you’re not allowed.”

03 · HTTP

HTTP/1.1 → HTTP/2 → HTTP/3

Version	Runs on	Big idea	Remaining pain
HTTP/1.1	TCP	One request at a time per connection; browsers open about 6 connections per site	Requests wait in line (head-of-line blocking)
HTTP/2	TCP	Many requests share one connection (multiplexing), binary, compressed headers	One lost TCP packet still stalls every stream
HTTP/3	QUIC over UDP	Independent streams; TLS built in, so setup is faster	Some networks block or slow UDP

Good news: your code rarely changes. The browser, CDN and server negotiate the version. Open DevTools → Network → add the **Protocol** column to see h2 or h3.

04 · HTTPS & TLS

HTTPS = HTTP inside a TLS tunnel

Privacy

Traffic is encrypted. Someone on café Wi-Fi cannot read your password.

Integrity

Nobody in the middle can change the page — no injected ads or scripts.

Identity

The certificate proves you are talking to the real robendevs.com.

Default port **443**. Browsers mark plain HTTP pages “Not secure”, and many features (geolocation, service workers, Secure cookies) need HTTPS.

The TLS 1.3 handshake, simplified

1. **Client Hello** — “I support these ciphers; here is my key share.”
2. **Server Hello + certificate** — “Let’s use this cipher; here is my key share and my ID card.”
3. **Browser checks the certificate** — trusted issuer, not expired, name matches.
4. **Same secret on both sides** — encrypted HTTP starts. Only 1 round trip.



Classic outage: a certificate expires at 2 AM and every user sees “Your connection is not private”. Fix: automate renewal (e.g. Let’s Encrypt) and alert before expiry.

05 · BROWSER INTERNALS

Fast network ≠ fast page.

The bytes have arrived. Now the browser must parse, style, lay out and paint — and that is the second half of the journey.

05 · BROWSER INTERNALS

A browser is a team of processes

Browser

Address bar, tabs, buttons. The manager.

Network

DNS, connections, HTTP cache.

Renderer

One per site. HTML, CSS, layout and the JavaScript engine. Sandboxed.

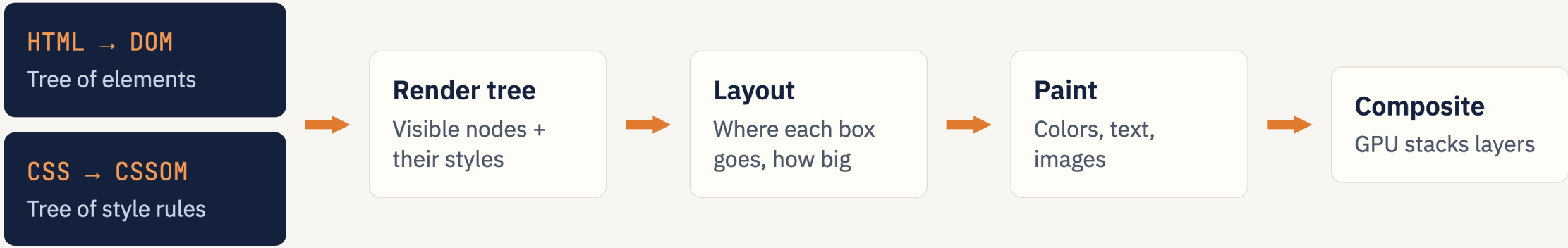
GPU

Draws the final pixels on screen.

Browser	Rendering engine	JavaScript engine
Chrome, Edge	Blink	V8
Safari	WebKit	JavaScriptCore
Firefox	Gecko	SpiderMonkey

Why separate processes? One crashing tab doesn’t kill the others, and a malicious site can’t read your files or other tabs.

From bytes to pixels: the rendering pipeline



If JavaScript changes...	The browser must redo
width, height, position	Layout → Paint → Composite (reflow, expensive)
color, background	Paint → Composite (repaint)
transform, opacity	Composite only (cheapest — use for animations)

What blocks the first paint

```
<head>
<link rel="stylesheet" href="app.css">
<script src="a.js"></script>
<script async src="analytics.js"></script>
<script defer src="app.js"></script>
</head>
```

Tag	What happens
CSS link	Blocks painting until the CSSOM is ready
plain script	Stops HTML parsing to download and run
async	Downloads in parallel, runs when ready, any order
defer	Downloads in parallel, runs after HTML is parsed, in order

Rule of thumb: small critical CSS first, **defer** for your app code, **async** for independent scripts like analytics.

06 · CACHING & SECURITY

Caching: the fastest request is the one you never send

Browser cache
On your laptop · 0 ms

→

CDN edge
A server near the user

→

Origin server
Your app · slowest

Header	Meaning
Cache-Control: max-age=31536000, immutable	Keep for a year — for files with a hash in the name
Cache-Control: no-cache	Store it, but check with the server every time
Cache-Control: no-store	Never store (bank statements, personal data)
ETag + If-None-Match	“Changed?” → 304 Not Modified, no body sent

Classic bug: “I deployed, but users still see the old site.” Use versioned file names like app.3f9a2c.js.

Cookies and browser storage

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax; Max-Age=3600
```

Attribute	Protects against
HttpOnly	JavaScript can’t read it (XSS theft)
Secure	Only sent over HTTPS
SameSite	Other sites can’t send it (CSRF)
Max-Age	How long it lives

Storage	Key facts
Cookie	~4 KB, sent with every request
localStorage	~5 MB, JS only, stays forever
sessionStorage	Per tab, gone when tab closes

For login sessions, prefer an **HttpOnly, Secure** cookie — any injected script can read localStorage.

Same-origin policy and CORS

Origin = **scheme + host + port**. Compared with **https://app.robendevs.com**:

URL	Same origin?
https://app.robendevs.com/settings	Yes
http://app.robendevs.com	No — scheme
https://api.robendevs.com	No — host
https://app.robendevs.com:8443	No — port

PREFLIGHT FOR PUT / JSON / CUSTOM HEADERS

```
OPTIONS /users/7
Origin: https://app.robendevs.com
Access-Control-Request-Method: PUT

204 No Content
Access-Control-Allow-Origin:
https://app.robendevs.com
Access-Control-Allow-Methods: PUT
```

Remember: CORS is enforced by the **browser**. Postman and curl ignore it. The fix is a header on the API, not in the frontend.

06 · CACHING & SECURITY

Three attacks every web engineer must know

</>

XSS

An attacker’s script runs inside your page — e.g. a comment containing <script>.

Defend: escape output, Content Security Policy, HttpOnly cookies

CSRF

Another site makes your browser send a request — with your cookies attached.

Defend: SameSite cookies, CSRF tokens

Man-in-the-middle

Someone on the network reads or changes your traffic.

Defend: HTTPS everywhere, HSTS header

Golden rule: **never trust input from the browser**. Validate on the server, always.

PUTTING IT TOGETHER

When it breaks: find the layer first

What the user sees	Likely step	First thing to check
DNS_PROBE_FINISHED_NXDOMAIN	DNS	dig the domain — is the record there?
ERR_CONNECTION_TIMED_OUT	TCP / firewall	Is port 443 open? Security group?
“Your connection is not private”	TLS	Certificate expired or wrong domain
502 Bad Gateway	Proxy → app	Is the app server running?
504 Gateway Timeout	App too slow	Slow database query or API call
“Blocked by CORS policy”	Browser	Allow-Origin header on the API
Users still see the old version	Caching	Cache-Control, versioned file names
Page loads but feels frozen	Rendering / JS	DevTools Performance: long tasks

TRY IT YOURSELF

See every step with your own eyes

`dig robendevs.com`

DNS answer and TTL (Windows: nslookup)

`curl -v https://robendevs.com`

Watch TCP connect, TLS handshake and HTTP headers

`curl -I https://robendevs.com`

Response headers only: cache, cookies, server

`DevTools → Network → reload`

Hover a request's waterfall: DNS, Connect, SSL, Waiting (TTFB), Download

`Click the padlock`

View the certificate, issuer and expiry date

`DevTools → Application`

Cookies with their flags, localStorage, cache

YOUR ASSIGNMENT

Research, draw, push to GitHub

`day-01-networking/`

Draw the full request path from browser to database and back.

Mark where TLS can be terminated and where responses can be cached.

`day-02-browser-internals/`

Explain why fast network requests do not guarantee a responsive page.

Diagram the DOM, CSSOM, layout and paint stages.

Each folder: notes.md, diagrams/, questions.md

```
docs(day-01): add request path diagram
```

Push by end of day and share the link. Write it in **your own words** — you'll defend it at checkpoint CP-1 on Day 3.

DISCUSSION

Think it through

1. Why does DNS mostly use UDP, while HTTP/1.1 and HTTP/2 use TCP?
2. Our site is fast in the USA but slow in Dhaka. Which steps would you check?
3. Postman can call our API, but the browser shows a CORS error. Why?
4. A certificate expires at 2 AM. What breaks, and how would we notice?
5. Why is animating transform cheaper than animating left?

Questions? “I don’t know yet, but I would check X” is a great answer.